

Reviewer Pack — Hashimoto Non-Backtracking Pole Gap Lower-Bounds Tseitin Res...

Entry 741fc0a4ec69 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 20:33:52 UTC

Hashimoto Non-Backtracking Pole Gap Lower-Bounds Tseitin Resolution

Entry ID: 741fc0a4ec69 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 20:33:52 UTC

1. Conjecture

Field A (mathematical branch): Ihara zeta functions of finite graphs (number-theoretic non-backtracking spectra, Bass-Hashimoto theory) **Field B** (complexity object): Resolution refutation length of Tseitin formulas $T(G, \sigma)$

Statement:

Let $G=(V,E)$ be any connected graph (possibly irregular) and B_G be its $2|E| \times 2|E|$ Hashimoto non-backtracking operator ($B_{\{(a,b),(c,d)\}}=1$ iff $b=c$ and $a \neq d$). Define the Ihara pole gap $\nu(G) := \log |\lambda_1(B_G)| - \log \max\{|\lambda| : \lambda \in \text{Spec}(B_G), \lambda \notin \{\pm \lambda_1, \pm 1\}\}$, set to 0 if no such second eigenvalue exists. Then for every odd-charge labeling $\sigma: V \rightarrow \{0,1\}$ with $\sum \sigma(v) \equiv 1 \pmod{2}$, the minimum Resolution refutation length $L(T(G, \sigma))$ satisfies $\log_2 L(T(G, \sigma)) \geq c \cdot |V| \cdot \nu(G)$ for an absolute constant $c > 0$, and $\nu(G) = O(1/|V|)$ on every graph admitting an $O(1)$ -vertex separator.

Rationale (proposer's reasoning):

Standard expansion arguments use λ_2 of the adjacency or Laplacian matrix, but the Bass-Hashimoto operator captures non-backtracking closed-walk multiplicities that better reflect how Resolution must trace clause-derivation paths without revisiting variables. Because B_G is the natural transfer operator of the Ihara zeta function, its second-pole gap is the right quantitative analogue of a 'prime number theorem' for graphs — and bottleneck graphs (long paths, dumbbells, weak cuts) automatically have collapsed gap. Importing zeta-function multiplicities into proof complexity is essentially uncharted (<5 papers) and avoids the algebrization barrier because B_G is a combinatorial integer matrix derived purely from the formula's underlying graph.

Taxonomy category: AVG_TO_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 502dd79eb39e146a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 6 graph families $\times$ 6 sizes $\times$ 30 seeds (1080 instances), regress $\log_2(\text{DPLL_nodes})$ on $n \cdot \nu(G)$; SUPPORTED iff OLS slope ≥ 0.05 with $R^2 \geq 0.6$ AND every dumbbell and path-of-triangles instance has $\nu(G) < 0.1 \cdot \log(n)$. FALSIFIED iff any single instance has $\nu(G) \geq 0.2$ yet $\log_2(\text{nodes}) < 0.01 \cdot n \cdot \nu(G)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Ihara zeta function non-backtracking spectrum expander Tseitin resolution lower bound - Hashimoto operator second eigenvalue Ramanujan graph proof complexity Tseitin formula - non-backtracking spectral gap resolution refutation length Tseitin expansion

Top relevant hits considered: - [http://arxiv.org/abs/math/0311029v1] Zeta functions over zeros of general zeta and L -functions - [http://arxiv.org/abs/1306.5454v1] The Ihara zeta function of the infinite grid - [http://arxiv.org/abs/1712.05169v12] Zeros and the functional equation of the quadrilateral zeta function - [http://arxiv.org/abs/2112.11882v2] Explicit values for Ramanujan's theta function $\varphi(q)$ - [http://arxiv.org/abs/0904.1226v1] On an Asymptotic Series of Ramanujan - [http://arxiv.org/abs/math/0604407v4] New Finite Rogers-Ramanujan Identities - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2403.19550v2] Spectral gap for surfaces of infinite volume with negative curvature

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def hashimoto_operator(graph):
    V, E = graph
    n = len(V)
    m = len(E)
    B = [[0] * (2*m) for _ in range(2*m)]

    for (a, b), (c, d) in E:
        if b == c and a != d:
            B[2*a][2*m + 2*b] = 1
            B[2*c][2*m + 2*d] = 1
```

```

    return B

def spectral_gap(B):
    eigenvalues = [complex(eigenvalue) for eigenvalue in numpy.linalg.eigvals(B)]
    lambda_1 = max(abs(eigenvalue) for eigenvalue in eigenvalues)
    other_eigenvalues = [eigenvalue for eigenvalue in eigenvalues if abs(eigenvalue) != lambda_1]
    if not other_eigenvalues:
        return 0
    lambda_max_other = max(abs(eigenvalue) for eigenvalue in other_eigenvalues)
    return math.log(lambda_1) - math.log(lambda_max_other)

def dpll_node_count(G, sigma):
    # Placeholder for DPLL node count estimation
    # This is a very rough approximation and should be replaced with actual logic
    n = len(G[0])
    m = len(G[1])
    return (n + m) * 2

def run_trial(seed: int) -> dict:
    random.seed(seed)

    graph_families = {
        "cycle": lambda n: ([i for i in range(n)] + [0], [(i, (i+1)%n) for i in range(n)]),
        "dumbbell": lambda n: ([i for i in range(n//2)] + [j+n//2 for j in range(n//2)],
                               [(i, i+1) for i in range(n//2-1)] + [(j+n//2, j+n//2+1) for j in range(n//2-1)]),
        "random_3regular": lambda n: random_regular_graph(n, 3),
        "random_4regular": lambda n: random_regular_graph(n, 4),
        "path_of_triangles": lambda n: ([i for i in range(3*n)], [(i, i+1) for i in range(3*n-1)]),
        "complete_bipartite": lambda n: ([i for i in range(n//2)] + [j+n//2 for j in range(n//2)],
                                         [(i, j+n//2) for i in range(n//2) for j in range(n//2)])
    }

    results = []
    for family_name, graph_func in graph_families.items():
        n_values = [8, 12, 16, 20, 28, 36]
        for n in n_values:
            G = graph_func(n)
            V, E = G
            sigma = {v: random.randint(0, 1) for v in V}

            B_G = hashimoto_operator(G)
            nu_G = spectral_gap(B_G)

            nodes = dpll_node_count(G, sigma)
            log_nodes = math.log2(nodes)

            results.append({
                "metric_name": "log2(DPLL_nodes)",
                "metric_value": log_nodes,
                "instances_tested": 1,
                "conjecture_holds": nu_G < 0.1 * math.log(n),
                "counterexample": ""
            })

    overall_conjecture_holds = all(result["conjecture_holds"] for result in results)

```

```

counterexample = "" if overall_conjecture_holds else "mapping_undefined"

return {
    "seed": seed,
    "metric_name": "log2(DPLL_nodes)",
    "metric_value": sum(result["metric_value"] for result in results) / len(results),
    "instances_tested": len(results),
    "conjecture_holds": overall_conjecture_holds,
    "counterexample": counterexample
}

def random_regular_graph(n, k):
    if n * k % 2 != 0:
        raise ValueError("n*k must be even")

    V = list(range(n))
    E = []
    degrees = [k] * n

    for i in range(k):
        available_nodes = set(V) - {i}
        while degrees[i] > 0:
            j = random.choice(list(available_nodes))
            if (j, i) not in E and (i, j) not in E:
                E.append((i, j))
                degrees[i] -= 1
                degrees[j] -= 1
                available_nodes.remove(j)

    return V, E

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_baef02fb.py", line 120, in <module>
    result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_baef02fb.py", line 69, in run_trial
    B_G = hashimoto_operator(G)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_baef02fb.py", line 24, in hashimoto_operator
    for (a, b), (c, d) in E:

```

^^^^^

TypeError: cannot unpack non-iterable int object

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `hashimoto_operator` before any data was produced, so the pre-registered regression and per-instance criteria cannot be evaluated. No RESULT line was emitted and the critic challenged on the same grounds. | next: Fix the Hashimoto operator construction to iterate over directed edge pairs as tuples (e.g., build E as a list of (u,v) tuples and index with enumerate) and rerun the full 1080-instance protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	149799
2	preregistration	claude_max	opus	0	0	7654
3	novelty	claude_max	opus	0	0	3598
4	novelty	claude_max	opus	0	0	8940
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	89759
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	190609
7	judge	claude_max	opus	0	0	5889

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 456249 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/741fc0a4ec69.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/741fc0a4ec69.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/741fc0a4ec69.tar.gz` (if generated)